BLACK CREATIVE

Référentiel pédagogique — Développement web

Sites responsive et initiation aux applications avec base de données

Formation professionnelle orientée métier • Public débutant • 6 mois • 144 heures encadrées

Version révisée 1.0 — Document de mise en œuvre pédagogique

1. Finalité et positionnement

Ce parcours prépare les apprenants à réaliser des sites web simples, à y ajouter des interactions et à participer à la réalisation d'une petite application connectée à une base de données. Il privilégie les gestes professionnels : comprendre un besoin, organiser son travail, produire, tester, corriger, documenter et livrer.

Le parcours constitue une première étape vers les activités d'intégration et de développement web junior. Il ne garantit pas une autonomie complète de développeur full-stack en 144 heures. La partie serveur, la base de données et le déploiement complet sont abordés sur un périmètre limité, avec documentation et accompagnement.

Public et prérequis

- Débutants en programmation disposant de bases d'utilisation d'un ordinateur.
- Savoir utiliser clavier et souris, naviguer sur Internet, créer et retrouver un dossier, déplacer des fichiers et extraire une archive.
- Aucun prérequis en programmation, en ligne de commande, en SQL ou en conception graphique.
- L'anglais technique est expliqué progressivement à partir des messages et termes effectivement rencontrés.

Diagnostic d'entrée et accompagnement

Le début de M1 comprend un diagnostic pratique non éliminatoire : créer un dossier, extraire une archive, retrouver un fichier, ouvrir une ressource et suivre une consigne courte. Le formateur repère les besoins d'accompagnement et utilise des fiches illustrées, des exercices gradués et les temps de correction intégrés aux modules. Si les bases informatiques sont insuffisantes pour suivre, le centre propose une remise à niveau distincte, annoncée comme telle ; elle n'est pas présentée comme incluse dans les 144 heures.

Compétences de sortie attendues

- **En autonomie sur un brief simple :** structurer et styliser un site vitrine responsive, vérifier sa navigation, appliquer les règles essentielles d'accessibilité et le publier selon une procédure connue.
- **Avec documentation :** ajouter des interactions JavaScript, traiter un formulaire, afficher des données, utiliser Git et diagnostiquer des erreurs courantes.
- **Avec accompagnement et supports fournis :** réaliser une API Express simple, la connecter à MySQL, vérifier les échanges et préparer une démonstration complète.
- **Dans tous les cas :** expliquer le code livré, signaler les limites du résultat et ne pas exposer de secrets ni de données sensibles.

2. Organisation et progression — 144 heures

Les contenus sont organisés par modules, non par semaines. À titre de repère, 144 heures correspondent à 24 semaines de 6 heures encadrées ; le calendrier réel est fixé par le centre.

Module	Intitulé	Durée
M1	Fondations du Web et environnement de travail	6 h
M2	HTML, CSS et responsive	24 h
M3	Projet 1 : site vitrine et publication statique	12 h
M4	Logique de programmation avec JavaScript	12 h
M5	JavaScript, DOM et interactions	24 h
M6	Initiation au back-end avec Node.js et Express	12 h
M7	Bases de données relationnelles et SQL	8 h
M8	Connexion Express–MySQL et intégration	12 h
M9	Tests, sécurité et qualité	6 h
M10	Déploiement guidé et procédure de démonstration	6 h
M11	Projet final : consolidation et recette	16 h
M12	Livraison, présentation et portfolio	6 h
Total		**144 h**

Principes pédagogiques

- Environ **70 % du temps encadré** est consacré à la manipulation, aux exercices, aux projets, au débogage et aux corrections actives. Ce temps est inclus dans les modules, et non ajouté aux 144 heures.
- Chaque notion suit une progression : démonstration courte, exercice guidé, exercice avec aide réduite, vérification individuelle.
- Les corrections, évaluations et reprises font partie des durées annoncées. Le formateur retire d’abord les extensions si le groupe a besoin de davantage de pratique.
- Une seule pile est enseignée : HTML, CSS natif, JavaScript, Node.js, Express et MySQL. Aucun framework front-end n’est obligatoire.
- Le projet fil rouge évolue pendant le parcours. M11 n’est pas le démarrage d’une application entièrement nouvelle.
- Des exercices courts dans d’autres contextes vérifient le transfert des acquis et évitent une dépendance au seul tutoriel.

Charge de travail et ressources

Le socle obligatoire est dimensionné pour les heures encadrées, avec les supports et fichiers de départ indiqués. Des exercices personnels facultatifs de consolidation peuvent être proposés. Tout travail extérieur rendu obligatoire doit être chiffré et annoncé séparément par le centre avant l’inscription ; il ne doit pas constituer une charge cachée.

Les fiches de consignes, maquettes, images autorisées, données fictives et supports essentiels sont téléchargeables pour limiter la dépendance à la connexion Internet.

3. Nouveau projet final — « Boutique Essentielle »

Catalogue et suivi simplifié des stocks d’une petite boutique

Situation professionnelle. Une petite boutique de fournitures souhaite présenter ses produits et disposer d’un outil simple pour actualiser leurs informations et leurs quantités disponibles. Les apprenants réalisent un prototype pédagogique, avec une identité de boutique fictive et des données de démonstration.

Utilisateurs. Un visiteur consulte le catalogue ; un gestionnaire met à jour les produits dans une interface locale ou protégée. Le projet ne gère ni clients, ni paiements, ni ventes.

Fonctionnalités obligatoires — socle commun

1. Une page d’accueil présentant la boutique, ses catégories et ses informations pratiques fictives.
2. Un catalogue responsive affichant les produits actifs : nom, catégorie, description courte, prix et disponibilité, sans obligation d’image par produit.
3. Un filtre par catégorie, réalisé côté navigateur sur les produits chargés.
4. Une interface de gestion listant tous les produits, y compris ceux qui ne sont plus affichés dans le catalogue.
5. Un formulaire pour créer un produit et modifier son nom, sa référence, sa catégorie, sa description, son prix, sa quantité et son état actif/inactif.
6. Une modification manuelle de la quantité disponible, avec refus des valeurs négatives et des valeurs non entières.
7. Une API Express et une base MySQL assurant la persistance des données.
8. Des messages compréhensibles pour les erreurs, le chargement, l’absence de résultats et les enregistrements réussis.
9. Un README, un script SQL, des données fictives, un plan de test et une démonstration reproductible.

Critères d’acceptation essentiels

- Le catalogue affiche les données issues de MySQL ; un produit inactif n’y apparaît pas.
- Le filtre fonctionne sans rechargement complet et indique l’absence de résultat.
- Un produit valide peut être créé puis modifié ; les changements restent visibles après rechargement.
- Une référence déjà utilisée, un champ obligatoire vide, une catégorie inconnue, un prix négatif ou une quantité invalide sont refusés côté serveur.
- Un arrêt de l’API produit un message utile, sans afficher de secret ni de détail interne sensible.
- La démonstration est locale, ou les accès de gestion sont effectivement protégés côté serveur avant exposition en ligne.

Extensions facultatives, sans pénaliser le socle

Recherche textuelle, indicateur de stock faible, pagination ou image choisie dans un jeu de fichiers fourni. Une seule extension est engagée à la fois, après validation du socle. Bootstrap ou Tailwind peuvent faire l’objet d’une découverte après consolidation du CSS natif, sans exigence de certification ni de livraison.

Hors périmètre

Paiement, facturation, panier, comptes clients, fournisseurs, import massif, téléversement de fichiers, multi-boutiques et historique des mouvements de stock. Les quantités sont actualisées manuellement : le prototype ne constitue pas un système de gestion commerciale prêt pour une exploitation réelle ou des modifications simultanées par plusieurs gestionnaires.

4. Projet final — architecture, sécurité et jalons

Modèle de données minimal

- **categories :** identifiant et nom. Les catégories sont fournies dans les données initiales ; leur gestion n’est pas obligatoire.
- **produits :** identifiant, catégorie liée, référence unique, nom, description courte, prix de type DECIMAL, quantité entière et état actif/inactif.
- Une catégorie peut regrouper plusieurs produits ; chaque produit est rattaché à une catégorie existante. Aucun stockage de données de clients n’est prévu.
- Le prix n’est pas utilisé pour des calculs de facturation. Le formateur fixe et documente les longueurs, les champs obligatoires et les valeurs autorisées.

Contrat d’API minimal

Route	Usage
GET /api/categories	Lire les catégories
GET /api/produits	Lire les produits actifs du catalogue
GET /api/gestion/produits	Lire tous les produits pour la gestion
POST /api/gestion/produits	Créer un produit
PATCH /api/gestion/produits/:id	Modifier les champs autorisés d’un produit

Les routes de gestion restent locales dans le scénario de base. Si l’application est exposée sur Internet, elles sont toutes soumises à authentification et autorisation côté serveur. Un endpoint DELETE peut être exercé sur des données jetables en travaux pratiques ; sa présence n’est pas exigée dans le projet final.

Deux modalités de livraison

A — Modalité de référence : démonstration locale. Application complète exécutée sur le poste, accès réseau extérieur désactivé, données fictives et procédure vérifiée sur un deuxième environnement préparé. Le site vitrine statique de M3 reste publié séparément. Aucun accès public à une gestion non protégée n’est autorisé.

B — Modalité en ligne, si les moyens sont disponibles. Hébergement préparé et validé par le formateur, HTTPS et protection serveur des routes de gestion. Le socle de protection peut être fourni ; l’apprenant doit savoir distinguer les accès publics et privés et vérifier un refus d’accès. Les mots de passe éventuels sont hachés, aucun secret n’est livré au navigateur. Si cette protection ne peut pas être vérifiée, retour à la modalité A.

Masquer un bouton, filtrer dans le navigateur ou configurer CORS ne constitue pas un contrôle d’accès. La construction autonome d’un système d’authentification n’est pas un objectif obligatoire de ce parcours.

Jalons et réutilisation

- **M2–M3 :** première version statique de la boutique, avec cartes et catalogue.
- **M4–M5 :** calculs simples, règles de validation, filtre et interactions sur données fictives.
- **M6–M7 :** API en mémoire puis création des deux tables.
- **M8 :** remplacement des données temporaires par la base et parcours complet de création/modification.
- **M9–M10 :** tests, vérifications de sécurité et procédure de livraison.
- **M11–M12 :** consolidation, recette, documentation et présentation individuelle.

M1 — Fondations du Web et environnement de travail

Durée : 6 heures • Prérequis : bases d’utilisation d’un ordinateur

Objectifs observables

- Identifier le rôle du navigateur, du serveur, de HTML, de CSS et de JavaScript.
- Organiser un dossier de projet et ouvrir une page dans le navigateur.
- Créer une première page HTML avec une feuille CSS externe.
- Enregistrer une première version avec Git en suivant une fiche de procédure.

Contenus prioritaires

- Diagnostic pratique d’entrée et accompagnement ciblé.
- Web et Internet ; parcours simplifié d’une URL ; requête et réponse ; différence HTTP/HTTPS. DNS est présenté par une illustration, sans approfondissement réseau.
- Éditeur, navigateur et outils développeur ; dossiers, extensions et chemins relatifs.
- Structure HTML minimale, titres, paragraphes, liens et images.
- CSS externe : sélecteurs simples, couleur, police et marges.
- Git : dépôt local, status, add et commit ; dépôt distant et premier envoi guidé. Création de compte accompagnée si nécessaire.

Exercices pratiques

1. Créer un dossier contenant une page, une feuille de style et un sous-dossier d’images.
2. Réaliser une page « Présentation de ma boutique » avec titre, texte, lien et image fournie.
3. Modifier son apparence et réparer un chemin d’image volontairement incorrect.
4. Enregistrer deux états du travail avec Git et rédiger un README de quelques lignes.

Livrables

Une page HTML/CSS fonctionnelle, un dossier organisé et un historique Git initial. Le dépôt distant est créé dès que les comptes et la connexion le permettent ; le travail local continue en cas d’indisponibilité.

Évaluation et remédiation

L’apprenant retrouve ses fichiers, montre la relation entre HTML et CSS, corrige un lien simple et enregistre une modification avec la fiche fournie. Le diagnostic n’est pas noté. En cas de difficulté, reprendre chemins et manipulations avant d’ajouter des notions.

Limites du module

Pas d’administration système, de configuration DNS, de JavaScript à écrire ni de maîtrise avancée de Git attendue. Le formateur dispose d’installateurs et de fichiers de départ testés.

M2 — HTML, CSS et responsive

Durée : 24 heures • Prérequis : M1

Objectifs observables

- Structurer une page avec des éléments HTML adaptés.
- Traduire une maquette simple en interface lisible sur mobile et ordinateur.
- Construire une mise en page avec Flexbox et une grille simple.
- Créer un formulaire correctement étiqueté et navigable au clavier.

Contenus prioritaires

- HTML sémantique : header, nav, main, section, article et footer ; liens, images et listes.
- Accessibilité : titres hiérarchisés, textes alternatifs utiles, labels, boutons appropriés et focus visible.
- CSS : cascade, héritage, spécificité sur des cas simples, modèle de boîte, unités, classes et états interactifs.
- Flexbox, Grid pour un catalogue, images fluides, media queries et démarche mobile-first.
- Lecture d’une maquette fournie : couleurs, tailles, espacements et alignements. Figma en lecture seule ou export PDF/image ; aucun apprentissage complet de l’outil n’est exigé.
- Réutilisation de classes, nommage clair et séparation des fichiers.

Exercices pratiques

1. Intégrer une fiche produit avec nom, description, prix et disponibilité.
2. Construire une navigation et une grille de produits adaptée à trois largeurs de contrôle.
3. Réaliser les champs du futur formulaire de gestion : nom, référence, catégorie, prix et quantité.
4. Reproduire une page simple à partir de la maquette fournie.
5. Tester clavier, titres, images, focus et absence de débordement horizontal ; corriger les défauts relevés.

Livrables

Une page de catalogue statique responsive, un formulaire visuel accessible et une feuille de style organisée. Ces éléments seront réutilisés en M3 et dans le projet final.

Évaluation et remédiation

L’apprenant réalise une modification de disposition sans recopier une page entière et explique le modèle de boîte sur un élément. La vérification porte sur la structure, la lisibilité, le responsive et les interactions clavier. Les erreurs fréquentes font l’objet d’un atelier de correction inclus dans les 24 heures.

Hors socle obligatoire

Bootstrap, Tailwind, animations élaborées, positionnement complexe et reproduction d’une maquette professionnelle volumineuse. Des transitions simples restent possibles lorsque les acquis prioritaires sont stabilisés.

M3 — Projet 1 : site vitrine et publication statique

Durée : 12 heures • Prérequis : M1–M2

Objectifs observables

- Transformer un brief court en un site cohérent de trois pages.
- Réutiliser les éléments produits en M2 et organiser une navigation commune.
- Tester, corriger et publier un site statique.
- Présenter le résultat et prendre en compte un retour utilisateur.

Contenus prioritaires

- Brief, public cible, contenus utiles et croquis de pages.
- Organisation des dossiers et cohérence des styles partagés.
- Pages : accueil, catalogue et informations pratiques/contact.
- Optimisation élémentaire des images et vérification des liens.
- Liste de tâches, commits descriptifs, README et retour par un pair.
- Publication avec un service statique choisi et testé par le formateur.

Exercices et réalisation

1. Préciser en une page l’identité fictive de la boutique, ses visiteurs et les informations à afficher.
2. Assembler les composants de M2 en trois pages cohérentes ; les textes et images de base sont fournis.
3. Tester la navigation, les liens, l’affichage mobile et le clavier, puis corriger au moins trois défauts identifiés ou issus d’une fiche d’entraînement.
4. Publier le site et vérifier le résultat depuis un autre appareil si disponible.
5. Faire une présentation courte et appliquer un retour pertinent.

Livrables

Site vitrine de trois pages, lien public, dépôt versionné, README court et checklist de contrôle. Ce site constitue la version statique du projet fil rouge, pas un projet indépendant à reconstruire ensuite.

Évaluation et remédiation

Vérifier la complétude, la navigation, le responsive, les liens et la publication. Chaque apprenant explique et modifie une partie de son code. La qualité visuelle est appréciée au regard de la maquette simple, et non d’un niveau de designer professionnel.

Limites et solution de repli

Aucun formulaire ne prétend envoyer un message sans traitement réel. La page contact présente des informations fictives ; si une maquette de formulaire est conservée, elle est explicitement signalée comme non fonctionnelle. En cas de panne externe de l’hébergement, une démonstration locale et la procédure préparée servent de preuve provisoire, sans pénalité liée à la panne.

M4 — Logique de programmation avec JavaScript

Durée : 12 heures • Prérequis : savoir éditer et tester des fichiers

Objectifs observables

- Décomposer un petit problème en entrées, traitement et sorties.
- Utiliser variables, comparaisons, conditions, boucles et fonctions simples.
- Lire une liste d’objets et retrouver une information.
- Reproduire une erreur, l’isoler et vérifier sa correction.

Contenus prioritaires

- Valeurs, variables const/let, chaînes, nombres, booléens et conversions simples.
- Opérateurs, comparaisons et conditions if/else.
- Boucle for et parcours d’une liste ; les variantes ne sont pas multipliées.
- Fonctions : paramètres, résultat retourné et différence entre retourner et afficher.
- Tableaux et objets représentant quelques produits.
- Trace d’exécution, console et tests sur des valeurs ordinaires et limites.

Le pseudo-code sert ponctuellement à expliquer une démarche. Les exercices sont rapidement traduits en JavaScript pour éviter un double apprentissage abstrait.

Exercices pratiques

1. À partir d’une quantité, afficher « disponible » ou « épuisé » ; refuser une quantité invalide.
2. Calculer un total indicatif à partir d’un prix et d’une quantité, sans logique de paiement.
3. Parcourir une liste et compter les produits disponibles.
4. Écrire une fonction qui vérifie qu’un nom, après retrait des espaces inutiles, n’est pas vide.
5. Corriger plusieurs erreurs préparées : conversion oubliée, condition incorrecte et valeur mal retournée.
6. Transférer une règle simple à un autre contexte, par exemple les places disponibles d’un atelier.

Livrables

Petite série de scripts commentés, jeu de valeurs de test et explication d’une démarche en langage courant puis en code.

Évaluation et remédiation

Un exercice individuel court vérifie la lecture d’une condition, l’écriture d’une fonction simple et la correction d’une erreur. L’aide documentaire est permise ; le niveau d’aide reçu est consigné. Les reprises portent d’abord sur variables, conditions et fonctions.

Hors socle obligatoire

Algorithmique avancée, récursivité, complexité, programmation orientée objet et formalismes mathématiques. Les cas métier simples remplacent les exercices sans lien avec l’usage du Web.

M5 — JavaScript, DOM et interactions

Durée : 24 heures • Prérequis : M2 et bases de M4

Objectifs observables

- Relier des actions utilisateur à des modifications de l’interface.
- Afficher et filtrer une liste de produits à partir de données.
- Valider un formulaire et afficher des messages utiles.
- Lire une réponse JSON avec fetch et traiter les principaux états de l’interface.

Contenus prioritaires

- Consolidation de fonctions, tableaux, objets et portée sur des exemples courts.
- DOM : sélectionner, créer et modifier des éléments ; utiliser textContent pour afficher du texte issu de données.
- Événements, formulaires, preventDefault, lecture des champs et messages associés.
- Validation client pour l’expérience utilisateur ; distinction explicite avec la validation serveur.
- JSON ; chargement d’un fichier de démonstration depuis un serveur local fourni, puis lecture d’une API pédagogique.
- Introduction ciblée à async/await, fetch, réponse HTTP en erreur, try/catch et état de chargement.
- Console, points d’arrêt et séparation HTML/CSS/JS.

Exercices pratiques

1. Afficher des cartes à partir d’un tableau de produits, sans recopier le HTML pour chaque produit.
2. Filtrer la liste par catégorie et afficher un message si elle est vide.
3. Valider nom, référence, prix et quantité ; conserver les valeurs saisies en cas d’erreur.
4. Simuler un enregistrement en mémoire et annoncer clairement la perte des modifications au rechargement.
5. Charger des données de démonstration et tester succès, réponse HTTP en erreur et indisponibilité.
6. Réaliser une petite interaction individuelle dans un autre contexte à partir d’une consigne nouvelle.

Livrables

Version interactive du catalogue, formulaire validé côté navigateur, données fictives et README indiquant que la persistance en base n’est pas encore réalisée.

Évaluation et remédiation

Vérifier le filtre, les champs invalides, les messages et la capacité à retrouver une erreur avec les outils développeur. L’apprenant explique une fonction et un gestionnaire d’événement. Le formateur fournit une aide graduée pour l’asynchronisme sans en faire un cours théorique complet.

Hors socle obligatoire

Framework JavaScript, classes avancées, programmation fonctionnelle approfondie et API publique instable. localStorage constitue uniquement une extension sur des données non sensibles ; aucun mot de passe ni secret n’y est stocké.

M6 — Initiation au back-end avec Node.js et Express

Durée : 12 heures • Prérequis : fonctions, objets, JSON et découverte de fetch

Objectifs observables

- Expliquer pourquoi le serveur intervient entre le navigateur et la base.
- Démarrer un petit serveur Express à partir d'une procédure connue.
- Créer des routes de lecture, création et modification sur des données en mémoire.
- Valider les entrées et vérifier les réponses avec un outil HTTP.

Contenus prioritaires

- Node.js, npm, package.json, scripts et dépendances ; une seule convention de modules est retenue.
- API, méthodes GET/POST/PATCH, paramètres, corps JSON et codes de statut courants.
- express.json, validation et gestion simple des erreurs.
- Organisation légère : point d'entrée et fichier de routes ; séparation supplémentaire seulement si elle clarifie le projet.
- Distinction entre routes publiques et routes de gestion ; les essais de gestion restent locaux.

Exercices pratiques

1. Démarrer un serveur et lire une liste de catégories et de produits en mémoire.
2. Créer les routes GET du contrat d'API ; n'exposer que les produits actifs dans le catalogue.
3. Créer POST /api/gestion/produits avec contrôle des champs obligatoires et des valeurs.
4. Créer PATCH /api/gestion/produits/:id avec liste explicite des champs modifiables.
5. Tester création valide, corps invalide, référence dupliquée et identifiant inexistant.

Livrables

API locale en mémoire, commandes de lancement, exemples de requêtes et tableau des résultats attendus. Le README précise que les données disparaissent au redémarrage.

Évaluation et remédiation

L'API renvoie du JSON, des codes cohérents et des erreurs lisibles sans planter. L'apprenant peut suivre le parcours requête–validation–réponse avec documentation. Le formateur peut fournir un squelette de serveur ; les routes et règles travaillées doivent être expliquées par l'apprenant.

Vigilances

Aucune connexion à MySQL à ce stade. Pas de multiplication des architectures ni de bibliothèques. Les routes de gestion non protégées ne doivent pas être publiées. Le fait qu'une interface masque les boutons ne protège pas ces routes.

M7 — Bases de données relationnelles et SQL

Durée : 8 heures • Prérequis : manipulation de données sous forme d’objets

Objectifs observables

- Représenter deux entités liées et expliquer les clés.
- Créer les tables categories et produits dans un environnement préparé.
- Lire, créer, modifier et supprimer des données de test avec SQL.
- Vérifier une contrainte d’unicité et une relation entre tables.

Contenus prioritaires

- Tables, colonnes, enregistrements et types utiles : entier, texte, DECIMAL et état actif.
- Clé primaire, clé étrangère et référence produit unique.
- CREATE TABLE, INSERT, SELECT, WHERE, ORDER BY, UPDATE et DELETE.
- Jointure simple pour afficher le nom de la catégorie d’un produit.
- Contraintes, données cohérentes et risque d’une modification sans WHERE.
- Principe des requêtes paramétrées, qui sera appliqué dans M8.

Exercices pratiques

1. Dessiner le modèle categories–produits et expliquer la relation.
2. Construire le script SQL avec une fiche de syntaxe et les types proposés.
3. Insérer quelques catégories et une dizaine de produits fictifs.
4. Afficher les produits d’une catégorie, les produits disponibles et les produits triés par nom.
5. Modifier une quantité ; supprimer uniquement un produit de test dans une copie de travail.
6. Tenter une référence dupliquée et une catégorie inexistante, puis expliquer les refus.

Livrables

Schéma simple, script de création, jeu de démonstration réinitialisable et six requêtes commentées, dont une jointure simple.

Évaluation et remédiation

L’apprenant retrouve une information avec SELECT, modifie l’enregistrement demandé sans toucher aux autres et explique les clés avec ses propres mots. La documentation SQL est autorisée. Les exercices utilisent une base de travail jetable.

Limites et préparation

Le formateur prépare et teste MySQL avant le module pour éviter de consacrer l’essentiel des huit heures à l’installation. Administration de serveur, optimisation avancée, procédures stockées et théorie approfondie de la normalisation sont hors socle. La jointure reste limitée au besoin réel du catalogue.

M8 — Connexion Express–MySQL et intégration

Durée : 12 heures • Prérequis : M5–M7

Objectifs observables

- Connecter l’API à MySQL à partir d’une configuration documentée.
- Remplacer les données en mémoire par des requêtes paramétrées.
- Réaliser un parcours complet entre formulaire, serveur et base.
- Vérifier la persistance et diagnostiquer une erreur simple à chaque niveau.

Contenus prioritaires

- Pilote MySQL choisi par le formateur, configuration et variables d’environnement.
- Fichier .env local, .gitignore et .env.example sans valeur secrète.
- Requêtes paramétrées ; séparation légère du code de connexion.
- Validation serveur : types, champs autorisés, limites, unicité et catégorie existante.
- Appels fetch pour lecture, création et modification ; vérification des réponses HTTP.
- Messages d’erreur utiles sans divulgation d’informations internes.

Exercices pratiques

1. Connecter GET /api/categories et GET /api/produits aux tables.
2. Brancher le catalogue sur l’API et vérifier le filtre côté navigateur.
3. Créer un produit depuis le formulaire et confirmer son existence dans la base.
4. Modifier sa quantité, puis son état actif ; vérifier les vues catalogue et gestion.
5. Redémarrer l’application et vérifier que les données sont conservées.
6. Simuler une mauvaise saisie, un doublon, une catégorie inconnue et une indisponibilité de la base.

Livrables

Prototype local connecté, routes documentées, script SQL à jour, configuration d’exemple et premiers scénarios de test de bout en bout.

Évaluation et remédiation

L’apprenant démontre au minimum un parcours complet de création puis modification avec persistance. Il sait rechercher si un défaut provient du navigateur, de la réponse API ou de la base, avec une checklist. Le formateur propose une version de référence pour débloquer une intégration, sans valider automatiquement la compétence de l’apprenant.

Limites

Pas d’ORM, d’architecture complexe, de gestion de transactions de vente ni de mouvements de stock. La suppression a déjà été exercée en SQL ; une route DELETE sur données jetables reste facultative. La version locale avec données fictives est la cible commune.

M9 — Tests, sécurité et qualité

Durée : 6 heures • Prérequis : prototype intégré de M8

Objectifs observables

- Définir et exécuter des scénarios manuels reproductibles.
- Documenter un défaut et vérifier sa correction.
- Contrôler les protections élémentaires introduites dans les modules précédents.
- Décider si une démonstration peut être exposée en ligne ou doit rester locale.

Contenus prioritaires

- Recette, résultat attendu/observé, cas nominal, cas limite et régression.
- Validation navigateur et serveur ; limites de la seule validation HTML.
- Injection SQL et requêtes paramétrées ; affichage prudent des textes avec textContent.
- Secrets, dépôt Git et messages d’erreur ; remplacer immédiatement un secret divulgué, pas seulement supprimer le fichier.
- Authentification et autorisation ; CORS n’est pas une protection d’accès.
- Revue de code ciblée et checklist de livraison.

Exercices pratiques

1. Écrire et exécuter 8 à 10 tests couvrant notamment le filtre, la création, la modification, les champs invalides, les doublons, la persistance et l’indisponibilité de l’API.
2. Tester l’API directement avec une entrée invalide pour vérifier que le serveur ne dépend pas des contrôles du navigateur.
3. Vérifier l’absence de secrets dans les fichiers suivis et revoir l’historique si un incident est suspecté.
4. Afficher une chaîne ressemblant à du HTML comme du texte, sans qu’elle soit interprétée.
5. Reproduire un défaut, rédiger ses étapes, corriger et rejouer le test.
6. Pour la modalité en ligne seulement, vérifier le refus d’accès aux routes de gestion sans autorisation et le fonctionnement avec le compte autorisé.

Livrables

Plan de test renseigné, rapport de défauts, checklist de sécurité et liste des limites restantes.

Évaluation et remédiation

L’apprenant fournit des preuves reproductibles et explique les correctifs. Les tests manuels documentés constituent l’exigence commune ; aucun framework de tests automatisés n’est obligatoire. Une démonstration de test automatisé peut être proposée après validation du socle.

Condition de diffusion

Une gestion publique non protégée, un secret exposé ou une configuration non maîtrisée impose de suspendre la mise en ligne et de corriger, ou de revenir à une démonstration locale. Une bonne note ne dispense jamais de cette vérification.

M10 — Déploiement guidé et procédure de démonstration

Durée : 6 heures • Prérequis : M3 et M8–M9

Objectifs observables

- Distinguer application statique, serveur applicatif et base de données.
- Préparer une installation reproductible dans l'environnement retenu.
- Vérifier les paramètres, les accès et les parcours après installation.
- Documenter les étapes et les principaux incidents rencontrés.

Contenus prioritaires

- Local et hébergé ; dépendances, scripts, versions testées et configuration.
- Relecture du site statique déjà publié en M3, sans refaire un second cours de publication.
- Démonstration par le formateur d'un déploiement Express–MySQL sur une solution choisie à l'avance.
- Variables serveur et configuration publique ; aucun secret de base transmis au navigateur.
- Choix privilégié d'un même serveur pour le front-end et l'API afin de simplifier les URL et éviter une complexité CORS inutile.
- Logs, erreurs de connexion, vérification après redémarrage et limites des hébergements.

Exercices pratiques

1. Repartir d'une copie propre du dépôt et suivre le README.
2. Installer les dépendances, initialiser la base, renseigner la configuration locale et démarrer l'application.
3. Vérifier catalogue, création, modification et persistance.
4. Faire suivre la procédure par un pair sur un environnement préparé, puis corriger les omissions.
5. Suivre la démonstration de déploiement complet ; si les moyens le permettent, appliquer la procédure sur l'hébergement préparé avec les protections requises.

Livrables

README d'installation, .env.example, script SQL, versions testées, checklist de démarrage et procédure de dépannage. Le site statique est publié ; l'application complète est au minimum reproductible localement.

Évaluation et remédiation

Le résultat est évalué sur la reproductibilité, la compréhension de la configuration et la qualité des vérifications. La modalité locale sécurisée et documentée n'est pas moins bien notée qu'une modalité hébergée ; les ressources financières et les pannes de service ne doivent pas déterminer la note.

Préparation et limites

Le centre prépare une seule solution Node.js–MySQL et une solution locale de repli. Si aucun hébergement n'est disponible, le formateur utilise une démonstration préparée du déploiement et explicite ce qui n'a pas été exécuté en ligne. Administration Linux complète, Docker, pipelines CI/CD et comparaison de nombreuses offres sont hors socle.

M11 — Projet final : consolidation et recette

Durée : 16 heures • Prérequis : travaux du fil rouge M1–M10

Objectifs observables

- Vérifier l’adéquation du prototype au besoin de la boutique.
- Prioriser et terminer les fonctionnalités obligatoires.
- Corriger les défauts et conduire une recette à partir de critères connus.
- Préparer une livraison explicable et reproductible.

Contenus prioritaires

- Reprise du brief initial, utilisateurs, limites et critères d’acceptation.
- Backlog court : à faire, en cours, à vérifier et terminé.
- Cohérence visuelle, responsive et parcours de gestion.
- Intégration des acquis ; revue ciblée avec le formateur ou un pair.
- Tests, corrections prioritaires et documentation au fil de l’eau.

Organisation indicative des 16 heures

- **2 h :** état des lieux, validation du périmètre et plan de travail.
- **8 h :** consolidation des fonctionnalités et remédiation technique.
- **4 h :** recette, correction et tests de non-régression.
- **2 h :** préparation de la livraison et démonstration intermédiaire.

Travaux à réaliser

1. Mettre à jour la fiche projet d’une page et les croquis des écrans effectivement livrés.
2. Établir une liste de 8 à 12 tâches priorisées ; traiter les défauts bloquants avant les extensions.
3. Vérifier un parcours complet : lire le catalogue, créer un produit, le modifier et constater la persistance.
4. Faire tester le résultat par un pair et corriger les défauts majeurs.
5. Compléter le README, les tests, les limites connues et la preuve de sécurité correspondant à la modalité choisie.

Livrables

Version candidate à la livraison, dépôt organisé, fiche projet, schéma de données, plan de test renseigné et documentation presque finalisée.

Évaluation et remédiation

Évaluer la progression, la priorité donnée au socle, les corrections et la compréhension. Une aide ciblée reste possible et est consignée. Une fonctionnalité obligatoire manquante est signalée et fait l’objet d’un rattrapage ; elle n’est pas discrètement reclassée comme facultative.

Cadre de travail

Le travail en binôme est possible, avec rotation des rôles et traces des contributions. Chaque apprenant doit manipuler et expliquer l’interface, une route et une requête simple. M11 consolide le travail réalisé pendant les modules ; il ne finance pas la construction d’un nouveau projet depuis zéro.

M12 — Livraison, présentation et portfolio

Durée : 6 heures • Prérequis : version candidate de M11

Objectifs observables

- Livrer un ensemble de fichiers cohérent et utilisable.
- Démontrer les fonctionnalités et les limites de la solution.
- Expliquer sa contribution et réaliser une petite modification individuelle.
- Présenter le projet dans un portfolio sans exagérer son niveau d’autonomie.

Contenus prioritaires

- Vérification finale du dépôt, des données fictives, de la configuration et du README.
- Présentation courte : besoin, utilisateurs, démonstration, choix simples et limites.
- Bilan individuel : compétences maîtrisées, aides utilisées et prochaines étapes.
- Portfolio : captures, contexte, fonctionnalités, technologies, rôle personnel et liens utiles.

Exercices et livraison

1. Exécuter une dernière fois la checklist de livraison depuis une installation propre préparée.
2. Faire relire le README ; retirer les comptes de test sensibles et toute information non destinée au public.
3. Présenter le projet en environ cinq minutes, avec questions et manipulation individuelle courte selon l’organisation du groupe.
4. Réaliser une petite modification ou corriger une erreur sous observation, avec documentation autorisée.
5. Publier ou préparer une fiche portfolio, en indiquant clairement s’il s’agit d’une démonstration locale et d’un projet pédagogique accompagné.

Livrables finaux

- Code versionné et historique de travail compréhensible.
- Site vitrine et lien publié lorsque le service est disponible.
- Application complète locale ou en ligne selon les conditions de sécurité.
- Script SQL et données de démonstration fictives.
- README : besoin, périmètre, prérequis, versions, installation, lancement, configuration, tests et limites.
- Fiche projet, schéma de données, résultats des tests et captures.
- Fiche portfolio et bilan individuel des compétences.

Évaluation et organisation

La démonstration collective ne remplace pas la vérification individuelle. Le centre dimensionne les passages à l’effectif ; certaines observations individuelles sont recueillies dès M11 pour maintenir une évaluation réelle dans les heures prévues. La rédaction du README et du portfolio commence avant M12.

Suites du parcours

Le bilan oriente vers la consolidation de JavaScript, la multiplication de projets courts, une pratique encadrée en situation professionnelle et, seulement après stabilisation des bases, l’apprentissage d’un framework ou d’un back-end plus avancé.

5. Évaluation des acquis et barème final

Évaluation continue

Chaque module produit une réalisation vérifiable et un retour précis. Les observations identifient le niveau d'aide : sans aide, avec documentation, avec guidage ou non réalisé. Elles servent à organiser les reprises et à établir le bilan individuel, sans transformer chaque exercice d'apprentissage en examen.

Les critères sont communiqués avant la réalisation. La documentation est autorisée ; l'apprenant reste responsable du code livré et doit pouvoir l'expliquer et le modifier. Un résultat obtenu avec un squelette fourni ne constitue pas, à lui seul, une preuve d'autonomie.

Barème du projet final — 100 points

Critère	Points	Éléments observés
Compréhension du besoin et organisation	10	Brief, périmètre, tâches et suivi
Interface, responsive et accessibilité	20	Lisibilité, mobile, navigation, labels et clavier
JavaScript et parcours fonctionnels	20	Filtre, formulaire, états et gestion des erreurs
API, SQL et persistance	20	Routes, validations serveur, requêtes et données conservées
Tests, sécurité et corrections	15	Scénarios, preuves, secrets, accès et défauts corrigés
Documentation et reproductibilité	5	Installation vérifiée, configuration d'exemple et limites
Explication et manipulation individuelles	10	Compréhension, contribution et modification simple
Total	**100**	

Règles d'application

- Les extensions ne compensent pas l'absence d'une fonction essentielle et ne sont pas exigées pour obtenir la totalité des points du socle.
- Une démonstration complète locale, sûre et reproductible permet d'obtenir tous les points de livraison.
- L'indisponibilité d'un service externe n'est pas assimilée à une erreur de l'apprenant ; les preuves alternatives sont prévues.
- Les contributions individuelles sont vérifiées, même en binôme ; la seule apparence du projet ou le nombre de commits ne suffit pas.
- Une grille précise pour chaque critère les résultats conformes, partiels et non démontrés. Le centre annonce avant le démarrage ses règles de validation et de rattrapage ; le présent référentiel n'affirme aucune reconnaissance officielle de diplôme.

Conditions de mise en ligne distinctes de la note

Les secrets doivent être absents, les routes de gestion doivent être protégées si elles sont publiques, et aucune donnée personnelle réelle n'est utilisée dans les démonstrations. Un défaut bloquant impose une correction ou un retour au mode local. Le jury peut évaluer les acquis sur une démonstration locale sans autoriser une publication risquée.

Rattrapage ciblé

Le retour indique précisément la compétence à retravailler, la preuve attendue et le niveau d'aide autorisé. Les reprises courtes s'appuient sur les temps intégrés aux modules. Si une remise à niveau ou une nouvelle session d'évaluation dépasse le volume prévu, le centre en annonce séparément les modalités.

6. Conditions de mise en œuvre et engagements pédagogiques

Moyens à préparer avant le démarrage

- Un poste utilisable par apprenant autant que possible ; les binômes ponctuels ne doivent pas supprimer le temps de manipulation individuelle.
- Un environnement commun testé : navigateur récent, éditeur, Git, version maintenue de Node.js, Express, pilote MySQL et MySQL. Les versions retenues sont inscrites dans les supports.
- Des droits d'installation ou une préparation des postes par le centre ; une procédure de réinitialisation et une sauvegarde des travaux.
- Une maquette simple, des textes, images autorisées, fichiers de départ, fiches de commandes et jeux de données fictifs.
- Des ressources consultables hors ligne pour les activités essentielles ; une solution aux accès GitHub intermittents.
- Un hébergement statique et, pour la démonstration complète en ligne, une solution Node.js–MySQL préparée avec budget et limites connus.

Accompagnement des débutants

Le formateur fractionne les consignes, fait verbaliser les démarches et montre comment rechercher une erreur. Il prévoit une fiche d'aide avant de fournir une solution complète. Les apprenants avancés approfondissent une extension ; le groupe n'accélère pas au détriment des bases des autres.

La lecture d'un message d'erreur, la consultation de documentation et la formulation d'une demande d'aide font partie des compétences professionnelles. Les termes techniques sont expliqués dans un glossaire alimenté progressivement.

Documentation, réutilisation et outils d'assistance

Les apprenants peuvent consulter documentation, exemples et outils d'assistance selon les règles du centre. Ils indiquent les principales ressources et aides utilisées, respectent les licences et vérifient le résultat. Ils ne transmettent à aucun service externe des secrets, des identifiants ou des données personnelles réelles. L'évaluation individuelle porte sur ce qu'ils comprennent et savent réellement modifier.

Gestes métier transversaux

- Reformuler une demande et distinguer indispensable et facultatif.
- Décomposer le travail en tâches courtes et communiquer un blocage.
- Faire une modification limitée, la tester puis l'enregistrer avec Git.
- Rapporter un bug avec étapes, résultat attendu et résultat observé.
- Livrer une procédure lisible et annoncer honnêtement les limites.

Checklist du centre avant ouverture de la session

1. Confirmer le niveau d'entrée et les modalités du diagnostic.
2. Tester les postes, installations, comptes et ressources locales.
3. Publier le calendrier, les 144 heures encadrées et toute charge extérieure distincte.
4. Valider le brief, les fonctionnalités obligatoires et les supports du projet fil rouge.
5. Fixer la modalité de livraison de référence : locale ; n'annoncer la mise en ligne complète que si les moyens et protections sont disponibles.
6. Préparer la grille individuelle, le barème, les reprises et l'organisation des présentations selon l'effectif.
7. Expliquer ce que la formation prépare à réaliser et ce qu'elle ne permet pas encore de garantir.

Principe directeur : moins de technologies survolées, davantage de réalisations comprises, testées et expliquées.